A Curiosity-Driven Agentic System for Knowledge Gap Identification and Targeted Exploration

Fredrik Paulin

February 10, 2026

Abstract

Large Language Models (LLMs) provide strong natural-language reasoning capabilities but often fail to represent uncertainty explicitly, leading to overconfident responses when information is missing. This whitepaper proposes an agentic, curiosity-driven system that integrates an LLM with a structured knowledge model to explicitly distinguish knowns, unknowns, and uncertainties. The system parses multi-modal inputs (text, images, structured data), identifies knowledge gaps against a grounded knowledge representation, and generates targeted research or action plans to reduce uncertainty via retrieval, experimentation, simulation, or inspection. A feedback loop updates the knowledge model with new findings to support continuous learning and iterative refinement. We describe subsystem interfaces, data flow, operational constraints, and evaluation protocols, and we expand concrete case studies across space exploration, scientific research, industrial automation, healthcare, and education.

Contents

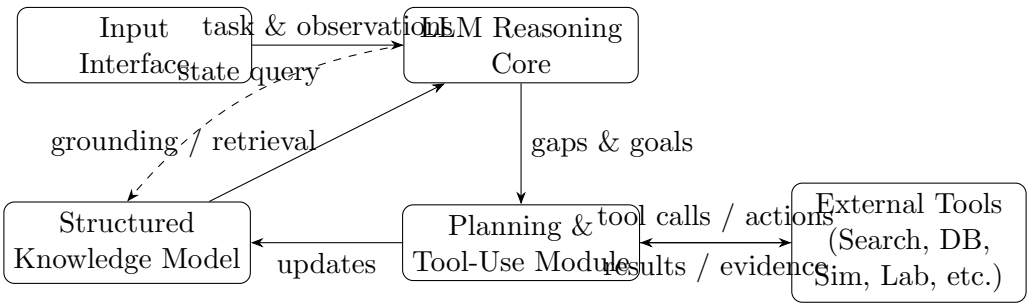

Figure 1: High-level architecture and information flow for a curiosity-driven agent: multi-modal inputs are parsed, grounded against a structured knowledge model, converted into gap-driven plans, executed via tools, and fed back to update knowledge state.

1 Introduction

Large Language Models (LLMs) can produce fluent, context-aware responses but do not reliably express the boundary between what is known, unknown, or uncertain. Identifying *knowledge gaps*, unanswered questions and missing causal or factual links, is fundamental to scientific inquiry and robust engineering decision-making [1]. Traditional approaches (e.g., manual literature review, expert elicitation, and ad-hoc testing) can be slow and difficult to scale [1]. In addition, LLMs can respond confidently even when the underlying truth is not available, leading to hallucinations and brittle later decisions [2].

This paper proposes an *agentic system* that operationalizes disciplined curiosity: it interprets a task, compares it against a structured knowledge model to identify gaps, and then plans and executes actions to reduce uncertainty. In contrast to a general-purpose conversational assistant, the agent treats "not knowing" as a first-class state, converts it into explicit investigative objectives, and closes the loop by updating its knowledge representation with validated findings [5].

The design uses an explicit known/unknown/uncertain partition aligned with the well-known "known knowns / known unknowns" framing [11]. The LLM provides a language and reasoning interface, while the surrounding system provides grounded state, disciplined planning, and verification loops that constrain failures such as hallucinated facts and untracked assumptions [2].

2 System Architecture Overview

The proposed architecture is modular and interface-driven, enabling the composition of multiple capabilities (retrieval, simulation, experimentation, and tool orchestration). Figure 1 shows the high-level data flow.

Core components:

- **Input Interface**: normalizes multi-modal inputs (text, images, structured data) into representations suitable for reasoning.
- **Structured Knowledge Model**: stores facts, relationships, evidence, and uncertainty state (known/unknown/uncertain).
- **LLM Reasoning Core**: performs task parsing, gap detection, and alignment between unstructured inputs and structured knowledge.
- **Planning and Tool-Use Module**: converts gaps into ordered actions (retrieval, simulation, experimentation, inspection).

- **Feedback and Learning Loop**: integrates new evidence, updates uncertainty, and triggers iterative re-planning.

2.1 System Interface: Multi-Modal Task Ingestion

The agent accepts tasks and observations in diverse formats:

- **Text**: user prompts, requirements, documents, procedures, and scientific papers.
- **Images**: photographs, microscopy images, satellite imagery, diagrams (optionally with OCR-derived text).
- **Structured data**: tables, time series, sensor logs, experiment results, or database records.

The interface transforms each modality into an internal representation suitable for reasoning. Typical mechanisms include metadata extraction, modality-specific feature extraction, and textual summarization. The primary design goal is *semantic normalization*: the LLM should receive a consistent schema of claims, observations, and contextual constraints, rather than raw heterogeneous data. Multi-modal agents increasingly rely on this pattern: a perception layer produces structured observations, which the reasoning layer uses to plan and query [5].

2.2 Knowledge Model: Representing Knowns, Unknowns, and Uncertainty

The structured knowledge model (SKM) grounds the agent's state. It can be implemented using:

- **Knowledge graphs / ontologies**: entities and relations with typed predicates.
- **Document + embedding stores**: unstructured text with semantic retrieval, optionally linked to a graph.
- **Probabilistic belief models**: explicit probability distributions or confidence scores per claim.
- **Digital twins**: structured, executable models of systems and their expected behavior [6].

Each knowledge element is annotated with:

- **State**: known, unknown, or uncertain (e.g., conflicting sources, low evidence, noisy measurements).
- **Evidence**: provenance, timestamps, measurement conditions, and validation status.
- **Confidence**: an interpretable scalar or distribution used for prioritization and verification.

This design treats missing information as explicit structure. In effect, the agent maintains not only a knowledge base but also an *ignorance-aware* model that tracks unanswered questions and weakly-supported assertions [4]. The LLM maps unstructured inputs into the SKM schema, acting as a structured extractor/classifier [7].

2.3 Role of the LLM: Task Parsing and Knowledge Gap Identification

The LLM performs two core functions: task parsing and gap identification.

Task parsing. Given a user goal, the LLM decomposes it into sub-questions, constraints, and success criteria. For technical tasks, this includes:

- identifying required variables and dependencies;
- extracting assumptions and boundary conditions;
- generating sub-goals aligned with the SKM schema.

Gap identification. For each sub-question, the LLM queries the SKM and assigns a state:

- **Known**: answer supported by stored evidence.
- **Unknown**: required information absent from SKM or retrieval fails.
- **Uncertain**: evidence exists but is weak, contradictory, or outside relevant operating conditions.

This step is designed to reduce hallucinations by forcing the model to explicitly separate retrieval-grounded knowledge from speculative inference. Uncertainty-aware prompting and calibration techniques are directly relevant here [2].

2.4 Planning Engine: From Gaps to Actionable Plans

The planning module converts gaps into actions that reduce uncertainty. Action classes include:

- **Information retrieval**: query internal corpora, databases, or curated knowledge sources.
- **Computation**: run analyses, statistical tests, or parameter sweeps.
- **Simulation**: invoke domain simulators or digital twins to test hypotheses safely [6].
- **Experiment design**: propose protocols, measurements, and controls.
- **Inspection / intervention**: recommend checks or operational steps (with safety constraints).

Plans must address:

- **Dependencies**: which gaps must be resolved first;
- **Cost and risk**: time, resources, safety, and expected value of information;
- **Verification**: how to validate each newly acquired fact before promotion to "known."

A practical prioritization heuristic is *information gain*: prefer actions expected to reduce uncertainty most per unit cost [9]. In safety-critical contexts, simulation-based validation (e.g., via a digital twin) provides a guardrail before physical actions [6].

2.5 Feedback Loop: Learning and Knowledge Model Update

After actions execute, results are interpreted and integrated:

1. **Interpretation**: parse tool outputs into SKM-compatible claims (with provenance).
2. **Validation**: cross-check sources, run consistency checks, and assign confidence.
3. **Update**: write new facts/relations; downgrade or flag inconsistent items; log uncertainty changes.
4. **Re-plan**: re-evaluate remaining gaps and iterate until the goal is met or blocked.

This closed-loop design implements curiosity as a controlled optimization process: repeated cycles reduce uncertainty and improve future planning. Without validation, error propagation is a primary failure mode; therefore, update policies must enforce evidence thresholds and provenance tracking [6, 2].

2.6 Implementation Considerations and Tech Stack (Architectural Level)

Although this paper remains architectural, implementation typically includes:

- **LLM orchestration**: structured prompting, tool calling, and state management; uncertainty-aware prompting/fine-tuning [2].
- **Knowledge representation**: hybrid graph + retrieval store; ontology for core entities; embeddings for fuzzy matching.
- **Tool adapters**: explicit interfaces for search, databases, simulation engines, and laboratory or operational systems.
- **State persistence**: versioned SKM updates, audit trails, and rollback for incorrect updates.

Bootstrapping the SKM may rely on initial extraction from corpora (technical docs, prior cases, papers), followed by iterative refinement as the agent encounters real tasks [13].

3 Applications and Case Studies Across Domains

3.1 Space Exploration and Planetary Science

In robotic exploration, autonomy is constrained by communication delays and limited bandwidth. A curiosity-driven agent can identify anomalies in imagery or spectroscopy, match them against known mineral libraries, and propose follow-up sensing or sampling plans. Related work includes rover autonomy that selects scientific targets and triggers instrument actions without immediate human direction [10]. The agent's value is in converting "this looks unusual" into an explicit gap (e.g., unknown composition) and an actionable, resource-aware plan (e.g., additional spectra, targeted drilling).

3.2 Scientific Research and Laboratory Automation

For literature review and hypothesis generation, the agent can ingest papers, extract claims and conditions into the SKM, detect inconsistencies or missing measurements (e.g., "no thermal data above 100°C"), and propose experiments or simulations to close those gaps [3, 5]. The key architectural requirement is rigorous provenance: each extracted claim must be tied to source, method, and context to prevent contamination of the SKM with misread or out-of-scope results.

3.3 Industrial Automation and Fault Diagnosis

Industrial processes increasingly use digital twins for monitoring and control. An agent can combine sensor streams with the twin model, identify deviations, and map them to known failure modes. When anomalies do not match known faults, the agent treats them as gaps and plans targeted diagnostics: log queries, tests, or simulation-based counterfactuals to isolate root cause [6]. Evaluation can measure time-to-diagnosis and correctness under novel fault types [8].

3.4 Healthcare: Diagnostic Planning and Knowledge Integration

In clinical decision support, missing information is common: incomplete labs, conflicting histories, or uncertain imaging findings. An ignorance-aware agent can represent differential diagnoses as hypotheses with uncertainty, identify which missing data would most reduce diagnostic ambiguity, and recommend tests or data integration steps. Safety constraints require strong verification and human oversight, explicit provenance, and conservative action selection; the architecture supports this via confidence thresholds and validation gates [2].

3.5 Education: Personalized Curriculum Planning

Intelligent tutoring systems model a learner's mastery and adapt instruction accordingly [12]. In this domain, "knowledge gaps" correspond to skills or concepts the learner has not mastered. The agent can maintain a SKM of concept dependencies, diagnose weak nodes from performance logs, and generate a learning plan (explanations, exercises, assessments). The feedback loop updates mastery estimates based on observed performance, enabling targeted remediation and curriculum refinement [12].

3.6 Other Domains

The same architecture applies to business intelligence (identifying missing market signals), cybersecurity (unknown attack paths requiring instrumentation), and engineering design (unknown failure modes requiring tests). Across domains, the common requirement is explicit uncertainty representation coupled with tool-grounded acquisition and validation.

4 Constraints and Challenges

Key constraints and failure modes include:

- **Hallucination and overconfidence**: mitigated via retrieval grounding, explicit gap state, and validation gates [2].
- **Error propagation**: incorrect updates can corrupt future reasoning; address with provenance, confidence, and rollback.
- **Ambiguity in mapping**: concept alignment between text and SKM can be lossy; require ontological discipline and disambiguation.
- **Scaling**: large SKMs and long-horizon plans require efficient retrieval, caching, and bounded context.
- **Safety**: especially for physical actions, incorporate simulation validation and human-in-the-loop approval [6].

5 Verification and Evaluation

We propose evaluation along four axes:

- **Gap detection accuracy**: agreement between agent-identified gaps and expert-labeled gaps.
- **Uncertainty reduction**: measure entropy/confidence improvement as actions execute [9].
- **Plan quality**: cost, time, risk, and informativeness (e.g., expected information gain).
- **End-to-end outcomes**: task success rate, time-to-resolution, and robustness under distribution shift [8].

A strong baseline is an LLM-only assistant without explicit SKM updates; improvements should be demonstrated in (1) fewer hallucinations, (2) faster convergence on correct decisions, and (3) better calibrated confidence reporting [2].

6 Conclusion

We described an architectural design for a curiosity-driven agentic system that integrates an LLM with a structured knowledge model to explicitly represent knowns, unknowns, and uncertainties. The agent converts knowledge gaps into targeted plans—retrieval, simulation, experimentation, or inspection—and closes the loop by updating the knowledge model with validated evidence. The system-of-systems design emphasizes modular interfaces, grounded state, and feedback-driven refinement to mitigate hallucinations and improve reliability. This architecture generalizes across high-value domains including space exploration, scientific discovery, industrial automation, healthcare, and education, where disciplined uncertainty reduction is a core operational requirement.